

Objective:

1. **Find out what the password is**
2. **Open the file** even though you don't know the password
3. Do this **legally** (because you're building a tool for police/forensics/Law) with authentication verification

TOOLS

These tools are like **digital keys** or **robots** that try many possible passwords until one works. Here's how each tool helps:

1. fcrackzip – For ZIP files (like .zip)

Use When: You want to open a ZIP file that asks for a password

What it does: Tries thousands of passwords quickly until one works

How it works:

- You give it a locked ZIP file
- You give it a list of possible passwords (called a "wordlist")
- It tries them one by one until it finds the correct one

2. pdftcrack – For PDF files (like locked question papers or reports)

Use When: You have a password-protected PDF file

What it does: Tries different passwords to open the PDF

Example:

- You give it a locked PDF and a wordlist
- It keeps trying passwords from the list until one works

3. john the ripper – Super tool that works with different file types

Use When: You want to unlock ZIP, Word, Excel, or PDF files

What it does:

- First: It **reads** the file and gets the password "fingerprint" (hash)
- Then: It **tries passwords** on that fingerprint
- If correct, it **shows the password**

Tech Stack

Feature	Tech Stack
UI	Streamlit (easy) or React
Backend	Python
Cracking	fcrackzip, pdfcrack, John the Ripper
File Upload	Python tempfile / Streamlit uploader
Auth	JWT / Firebase Auth
Storage	Local (encrypted) or AWS S3
Logging	Python logging or SQLite
Reports	ReportLab or HTML+CSS

PROJECT NAME: Digital Forensic Password Recovery & Evidence Tool

📌 **GOAL:** A secure web tool that allows authorized law enforcement personnel to upload password-protected files and retrieve the passwords using ethical AI-aided techniques, logging the activity for legal accountability.

🧩 STAGE 1: Planning & Use Case Design

1. 🎯 Define Scope:

- **Target users:** Only authorized officers from law enforcement or forensic teams.
- **File types:** ZIP, PDF, DOCX, XLSX, 7Z, RAR.
- **Cracking methods:** Dictionary attack, brute force, AI-guided word prediction.

2. Legal Considerations:

- **Clearly define user roles and responsibilities.**
 - **Use digital signatures, logs, and user verification to ensure lawful usage.**
 - **Avoid giving passwords directly to users — instead, show only file content in secure viewer unless admin allows export.**
-

STAGE 2: Authentication & Role-Based Access Control (RBAC)

1. Login Page

- **Email/ID + Password login**
- **2FA via OTP or organization email (optional)**
- **Google Workspace or Microsoft Entra ID (formerly Azure AD) for org login**

2. Verification via Org ID:

- **Use domain-restricted login (e.g., only @cybercrime.gov.in can register)**
- **Or integrate with organization SSO**

3. Roles & Access

- **Admin:** Full access, can see decrypted data
- **Forensic Analyst:** Can decrypt & view files but not export passwords
- **Observer:** Can view logs only

4. RBAC Implementation:

- FastAPI + OAuth2 with JWT tokens
 - Middleware to check roles before every route
-

STAGE 3: User Interface (Frontend)

Use React or Streamlit for simplicity.

Pages:

- Login Page (with org check)
- Upload Interface (drag and drop files)
- Case Status Dashboard (progress of password recovery)
- Secure Viewer (if password cracked)
- Logs and Activity History

 Tools: ReactJS or Streamlit + Tailwind CSS for styling

STAGE 4: Backend and File Handling

File Upload + Handling

- Store uploaded file temporarily using Python tempfile
- Detect file type using file signature (magic numbers)

Cracking Engine

- Use Python to run command-line tools like:
 - fcrackzip (ZIP)
 - John the Ripper (PDF, Office)

- msoffcrypto-tool (DOCX/XLSX)
- 7z via py7zr or subprocess

AI-Assisted Cracking:

- GPT model or BERT can suggest probable words from user context or metadata (e.g., file name, org info)
- Fine-tuned password generation using character-level LSTM or transformers

Example:

- GPT takes hints: filename, document title, user role
- Outputs likely passwords → try them first before brute-force

STAGE 5: AI Integration (Optional Advanced)

1. Train/Use ML model:

- Inputs: filename, metadata, known passwords
- Output: predicted password tokens
- Tools: HuggingFace Transformers or OpenAI GPT via API

2. Password Generation AI:

- Build a small model trained on leaked password patterns
- Suggest likely combinations, reducing brute force time

Warning:

- DO NOT use AI to guess user credentials or real-time systems.
 - Use strictly for offline encrypted files under legal scope.
-

STAGE 6: Logging, Storage & Reporting

1. Database:

- Use MySQL or PostgreSQL to store:
 - User ID
 - File info (type, size, name)
 - Cracking method used
 - Cracked password (optional)
 - File decrypted status
 - Timestamp

2. Logs:

- Use Python logging module + save to DB
- Log all attempts and file views

3. Reports:

- Auto-generate PDF reports (via ReportLab or WeasyPrint)
 - Contain metadata, attempt time, cracked password, etc.
-

STAGE 7: Deployment & Security

1. Deployment:

- Backend: FastAPI (Python)
- Frontend: React or Streamlit
- Host: AWS EC2 or Heroku (secure cloud hosting)
- Storage: AWS S3 or encrypted local folders

2. Encryption:

- Store cracked data using Fernet or AES encryption
- Password-protect download links (admin only)

3. Docker:

- Use Docker to containerize the tool for installation by different forensic agencies

STAGE 8: Final Questions & Safety

Should I show the cracked password?

- Only Admins should see the decrypted password.
- Others should only see the file content in a secure viewer (PDF/image/etc).
- Keep audit logs of all views.

Failsafes:

- Automatically delete files after decryption or timeout
- Log IP, login ID, and time of every decryption
- Auto-block if repeated failed attempts or misuse

TOOLS TO USE

Function	Tool
Frontend UI	Streamlit or React.js

Function	Tool
----------	------

Backend API	FastAPI
-------------	---------

Auth & RBAC	JWT + Firebase Auth or OAuth2
-------------	-------------------------------

Cracking Tools	fcrackzip, John the Ripper, py7zr, msoffcrypto
----------------	--

AI Cracking	Python + transformers or GPT-based suggestions
-------------	--

Database	MySQL or PostgreSQL
----------	---------------------

Storage	AWS S3 or secure filesystem
---------	-----------------------------

Logging	Python logging + SQL
---------	----------------------

PDF Reports	ReportLab, WeasyPrint
-------------	-----------------------